AIML ENGINEERING LAB — P053 REFERENCE GUIDE

Big Data Concepts End-to-End Guide for ML Engineers

Kafka, Spark, Airflow, and Everything In Between.
With P053 Production Examples.

Last Updated: April 2026 · Apache Kafka 3.x · Apache Spark 3.x · Apache Airflow 2.x

Table of Contents

1. What is "Big Data"? The 5 V's and Why ML Engineers Care

2. Data Storage — Files, Databases, and Data Lakes

- 2.1 File Formats (CSV, Parquet, ORC, Avro, Delta Lake)
- 2.2 Databases (PostgreSQL, MongoDB, Redis, DynamoDB)
- 2.3 Data Lakes vs Data Warehouses vs Lakehouses

3. Apache Kafka — Real-Time Data Streaming

- 3.1 Architecture (Brokers, Topics, Partitions, Consumer Groups)
- 3.2 When to Use Kafka (and When NOT to)
- 3.3 P053 Kafka Usage — Streaming Sensor Data

4. Apache Spark — Distributed Data Processing

- 4.1 Architecture (Driver, Executors, DAG, Stages)
- 4.2 Spark SQL, DataFrames, and RDDs
- 4.3 Spark MLlib — Distributed ML Training
- 4.4 P053 Spark Usage — ETL on 200M Rows
- 4.5 Why Pandas Fails at Scale (Show the Pain)

5. Apache Airflow — Workflow Orchestration

- 5.1 Architecture (Scheduler, Workers, DAGs, Operators)
- 5.2 Writing DAGs (Python, not YAML)
- 5.3 P053 Airflow DAGs — 3-DAG Architecture
- 5.4 Alternatives (Prefect, Dagster, Luigi, Argo)

6. MLflow — Experiment Tracking and Model Registry

- 6.1 Tracking (Parameters, Metrics, Artifacts)
- 6.2 Model Registry (@champion, @challenger)
- 6.3 P053 MLflow Architecture (PostgreSQL + S3)

7. Docker and Kubernetes for ML

- 7.1 Docker Compose (Multi-Container Applications)

7.2 Kubernetes (K8s) Orchestration

7.3 NVIDIA Container Toolkit (GPU in Docker)

8. Monitoring and Observability

8.1 Prometheus (Metrics Collection)

8.2 Grafana (Visualization and Alerting)

8.3 Model Drift Detection

9. Data Pipeline Patterns

9.1 Batch vs Stream vs Lambda vs Kappa

9.2 ETL vs ELT

9.3 Data Quality (Great Expectations, dbt)

10. Full Technology Comparison Matrix

11. P053 Complete Big Data Architecture

12. Interview Questions with Answers

1. What is "Big Data"? The 5 V's and Why ML Engineers Care

V	Definition	P053 Example
Volume	Amount of data (TB, PB)	16M rows for training + 200M rows over 40-day simulation = ~5B cumulative rows
Velocity	Speed of data generation	5M-10M rows/day in production (streaming via Kafka)
Variety	Different data types	Sensor data (numeric), spatial data (coordinates), temporal data (timestamps), images (wafer maps)
Veracity	Data quality/trustworthiness	5% label noise in synthetic data (simulating real fab conditions)
Value	Business insight from data	Each caught defect = $45K saved. 17.3% recall = $29M annual savings

Why ML engineers need big data skills: A model is only as good as the data pipeline feeding it. At companies like AMD, Micron, or NVIDIA, the data engineer and ML engineer are often the SAME person at the principal level. You're expected to design the pipeline end-to-end: ingest → transform → train → deploy → monitor.

2. Data Storage — Files, Databases, and Data Lakes

2.1 File Formats

Format	Type	Pros	Cons
CSV	Row-based	Human-readable, universal	No schema, no compression, slow for analytics
Parquet	Columnar	Compressed, fast analytics, schema embedded. **Used in P053**	Not human-readable
ORC	Columnar	Optimized for Hive/Spark, ACID transactions	Hive ecosystem lock-in
Avro	Row-based	Schema evolution, compact binary. Good for Kafka	Slower for analytics than columnar
Delta Lake	Parquet+	ACID on data lakes, time travel, schema enforcement	Databricks ecosystem
NPZ	NumPy	Fast load for numeric arrays. **Used in P053 training data**	NumPy-only, no schema

> **P053 data flow:** Raw data generated as NumPy → saved as NPZ for training (2.1 GB) → streaming via Kafka as Avro/JSON → Spark ETL writes Parquet to S3 → model reads Parquet for batch inference and drift detection.

```
# CSV vs Parquet comparison (actual P053 data)
import pandas as pd
import time

# CSV: 16M rows × 36 columns
t0 = time.time()
df = pd.read_csv('dram_16m.csv')  # 45 seconds, 8.2 GB RAM
print(f"CSV: {time.time()-t0:.1f}s, {df.memory_usage().sum()/1e9:.1f} GB

# Parquet: Same data
t0 = time.time()
df = pd.read_parquet('dram_16m.parquet')  # 3.2 seconds, 1.8 GB RAM
print(f"Parquet: {time.time()-t0:.1f}s, {df.memory_usage().sum()/1e9:.1f

# CSV file: 4.5 GB on disk
# Parquet file: 890 MB on disk (5x smaller, Snappy compression)
```

2.2 Databases

Database	Type	Best For	P053 Usage
PostgreSQL	SQL (RDBMS)	Structured data, ACID transactions, complex queries	✅ MLflow tracking backend (on RDS)
MySQL	SQL (RDBMS)	Web apps, read-heavy workloads	❌ Not used
MongoDB	NoSQL (Document)	Semi-structured data, JSON documents	❌ Not needed (data is tabular)
Redis	NoSQL (Key-Value)	Caching, real-time features, pub/sub	✅ Feature cache for inference API
DynamoDB	NoSQL (Key-Value)	Serverless, auto-scaling, low latency	❌ Not used (overkill for our scale)
Elasticsearch	Search engine	Full-text search, log analytics	❌ Not used
ClickHouse	Columnar OLAP	Real-time analytics on billions of rows	❌ Not used (Spark handles analytics)

2.3 Data Lakes vs Data Warehouses vs Lakehouses

Concept	What It Is	Examples
Data Lake	Store everything (raw, unstructured). Schema-on-read. Cheap.	S3, ADLS, GCS. **P053 uses S3**
Data Warehouse	Structured, cleaned data. Schema-on-write. Expensive.	Snowflake, BigQuery, Redshift
Data Lakehouse	Best of both: raw storage + SQL + ACID transactions	Delta Lake, Apache Iceberg, Apache Hudi

3. Apache Kafka — Real-Time Data Streaming

Apache Kafka is a distributed event streaming platform. Think of it as a real-time message highway — producers send messages, consumers read them, and Kafka stores everything durably in between.

3.1 Architecture

Component	Role
Broker	A Kafka server that stores and serves messages. Multiple brokers = a cluster.
Topic	Named channel for messages (like a table in SQL). P053: `wafer-sensor-data`
Partition	A topic is split into partitions for parallelism. Each partition is an ordered log.
Producer	Sends messages to a topic. P053: Data generator pushes sensor readings
Consumer	Reads messages from a topic. P053: Spark Streaming reads sensor data
Consumer Group	Multiple consumers sharing topic partitions. Load balancing.
Offset	Position of a consumer in a partition. Consumers track where they left off.
Zookeeper / KRaft	Cluster coordination. KRaft (Kafka Raft) replaces Zookeeper in Kafka 3.x+

```
# Python Kafka producer (conceptual - P053 uses Confluent Kafka)
from confluent_kafka import Producer

producer = Producer({'bootstrap.servers': 'kafka:9092'})

# Send wafer sensor data
for wafer_id in range(5_000_000):
    data = generate_wafer_data(wafer_id)
    producer.produce(
        topic='wafer-sensor-data',
        key=str(wafer_id),
        value=json.dumps(data)
    )
producer.flush()

# Python Kafka consumer
from confluent_kafka import Consumer

consumer = Consumer({
    'bootstrap.servers': 'kafka:9092',
    'group.id': 'spark-etl-group',
    'auto.offset.reset': 'earliest'
})
consumer.subscribe(['wafer-sensor-data'])

while True:
    msg = consumer.poll(1.0)
    if msg is not None:
        process(json.loads(msg.value()))
```

3.2 When to Use Kafka (and When NOT to)

✅ Use Kafka When	❌ Don't Use Kafka When
Real-time data needs (< 100ms latency)	Simple batch processing (use Airflow + S3)
Multiple consumers need the same data	One-time data migration
Decoupling producers from consumers	Data fits in a single database
Event sourcing / audit logging	Simple request-response (use REST API)
High throughput (millions of events/sec)	Small dataset (< 10K records/day)

3.3 P053 Kafka Usage

P053 Pipeline: Kafka simulates a real semiconductor fab's data flow.

- **Topic:** `wafer-sensor-data` — sensor readings from 33 process steps

- **Volume:** 1M-10M messages/day (variable, simulating production variance)
- **Format:** JSON messages with schema (wafer_id, features, timestamp, lot_id)
- **Consumer:** Spark Structured Streaming reads batches every 5 minutes
- **Retention:** 7 days (allows replay for debugging)

4. Apache Spark — Distributed Data Processing

Apache Spark is a distributed computing engine for large-scale data processing. It's 10-100× faster than Hadoop MapReduce because it processes data in-memory.

4.1 Architecture

Component	Role
Driver	Your main program. Creates SparkContext, builds execution plan, coordinates work.
Cluster Manager	Allocates resources (YARN, Mesos, K8s, or Standalone).
Executor	Worker JVM on each node. Runs tasks, stores cached data.
Task	Unit of work on a data partition. Executor runs many tasks in parallel.
DAG	Directed Acyclic Graph of operations. Spark builds this from your code.
Stage	Group of tasks that can run without shuffling data.
Shuffle	Data redistribution across nodes (expensive — avoid when possible).

4.2 Spark SQL, DataFrames, and RDDs

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("P053-ETL") \
    .config("spark.executor.memory", "4g") \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

# Read Parquet (columnar, pushdown predicates)
df = spark.read.parquet("s3://p053-mlflow-artifacts/data/day_20/")

# Transformations (lazy — nothing executes yet)
processed = (
    df
    .filter(F.col("wafer_temp") > 25)         # Filter rows
    .withColumn("temp_leak_ratio",             # Feature engineering
                F.col("wafer_temp") / F.col("leakage_current"))
    .groupBy("lot_id")                         # Aggregation
    .agg(
        F.avg("yield_prediction").alias("avg_yield"),
        F.count("*").alias("wafer_count"),
        F.stddev("leakage_current").alias("leak_std")
    )
)

# Action (THIS triggers execution)
result = processed.collect()  # Brings data to driver
processed.write.parquet("s3://p053-mlflow-artifacts/data/day_20_processe

# SQL interface
df.createOrReplaceTempView("wafers")
spark.sql("""
    SELECT lot_id, AVG(yield_prediction) as avg_yield
    FROM wafers WHERE wafer_temp > 25
    GROUP BY lot_id HAVING COUNT(*) > 100
""").show()
```

> **Lazy evaluation:** Spark doesn't execute anything until you call an **action** (`.collect()` , `.count()` , `.write()`). Transformations (`.filter()` , `.withColumn()`) just build the execution plan. This lets Spark optimize the entire pipeline before running it.

4.3 Spark MLlib — Distributed ML Training

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import VectorAssembler

# Assemble features into a single vector column
assembler = VectorAssembler(
    inputCols=["wafer_temp", "leakage_current", "timing_margin"],
    outputCol="features"
)

# Spark ML expects (features_vector, label) format
df_ml = assembler.transform(df).select("features", "label")

# Distributed logistic regression
lr = LogisticRegression(maxIter=100, regParam=0.01)
model = lr.fit(df_ml)
predictions = model.transform(df_ml)

# P053 note: We DON'T use Spark MLlib for the main model
# (PyTorch HybridTransformerCNN can't be distributed via Spark).
# Spark handles ETL; PyTorch handles training on a single GPU.
```

4.4 P053 Spark Usage

P053 Spark pipeline (in dag_daily_yield_pipeline.py):

1. **Ingest:** Read from Kafka topic → Spark Structured Streaming
2. **ETL:** Clean nulls, normalize features, compute engineered features
3. **Feature store:** Write processed Parquet to S3
4. **Drift detection:** Compute PSI, KL divergence, KS test on feature distributions
5. **Batch predict:** Load PyTorch model → predict on processed data

Volume: 5M rows/day × 40 days = 200M total rows processed

4.5 Why Pandas Fails at Scale (Show the Pain)

Metric	Pandas	Spark	Improvement
Load 16M rows CSV	45s, 8.2 GB RAM	3.2s (Parquet), distributed	14× faster
100M rows	❌ OOM (kills process)	✅ 12s on 4 executors	Pandas can't
GroupBy + Agg	2.1s (16M rows)	0.8s distributed	2.6×
Feature engineering	Single-threaded	Parallel across partitions	N× (N workers)
Max dataset size	~RAM/2 (8GB on 16GB)	Unlimited (spills to disk)	∞

> **Interview answer:** "Before introducing Spark, I demonstrated pandas failing on 100M rows with an OOM kill. The same ETL pipeline in Spark processes 200M rows in 12 seconds across 4 executors. A principal engineer doesn't just pick Spark because it's trendy — you show the PAIN of the alternative, then the solution."

5. Apache Airflow — Workflow Orchestration

Apache Airflow orchestrates complex data pipelines. It schedules tasks, manages dependencies, retries failures, and provides a visual UI.

5.1 Architecture

Component	Role
Scheduler	Parses DAG files, creates task instances, assigns to workers. Runs continuously.
Webserver	UI for monitoring DAGs, viewing logs, triggering runs
Worker	Executes tasks (CeleryExecutor, KubernetesExecutor, or LocalExecutor)
Metadata DB	PostgreSQL storing DAG state, task history, connections, variables
DAG	Directed Acyclic Graph — defines tasks and their dependencies in Python
Operator	Template for a task: PythonOperator, BashOperator, SparkSubmitOperator, etc.
Sensor	Waits for an external condition (S3 file exists, API returns 200)
XCom	Cross-communication — tasks share small data (< 48KB) between them

5.2 Writing DAGs (Python, not YAML)

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.utils.dates import days_ago
from datetime import timedelta

# DAG definition
with DAG(
    dag_id='p053_daily_yield',
    schedule_interval='@daily',
    start_date=days_ago(1),
    default_args={
        'retries': 2,
        'retry_delay': timedelta(minutes=5),
        'email_on_failure': True,
    },
    catchup=False,
    max_active_runs=1,
    tags=['p053', 'production'],
) as dag:

    # Task 1: Generate data
    generate = PythonOperator(
        task_id='generate_daily_data',
        python_callable=generate_daily_data,
        op_kwargs={'rows': 5_000_000},
    )

    # Task 2: Spark ETL
    etl = PythonOperator(
        task_id='spark_etl',
        python_callable=run_spark_etl,
    )

    # Task 3: Drift detection
    drift = PythonOperator(
        task_id='detect_drift',
        python_callable=run_drift_detection,
    )

    # Task 4: Batch inference
    predict = PythonOperator(
        task_id='batch_predict',
        python_callable=run_batch_inference,
    )

    # Dependencies
    generate >> etl >> [drift, predict]
```

```
#           etl ——> drift
# generate ——> etl ——> predict
```

5.3 P053 Airflow DAGs — 3-DAG Architecture

P053 uses 3 DAGs on AWS g4dn.xlarge:

DAG	Schedule	Tasks
`dag_simulation_master`	Once (manual trigger)	Orchestrates 40-day simulation, variable daily volumes, triggers daily DAG for each day, auto-stop EC2 at end
`dag_daily_yield_pipeline`	Triggered per day	Generate data → Kafka → Spark ETL → drift check → batch predict → log metrics → S3 upload
`dag_retrain_pipeline`	Triggered by drift	Real GPU training on T4 → canary test → promote/rollback in MLflow Registry → S3 upload

5.4 Alternatives

Tool	Key Difference from Airflow	When to Choose
Prefect	Python-native, no DAG files, dynamic workflows	Modern teams, simpler setup
Dagster	Software-defined assets, strong typing, dev experience	Data engineering teams
Luigi	Spotify's tool, simpler but less powerful	Legacy projects
Argo Workflows	Kubernetes-native, YAML-based	K8s-heavy orgs
AWS Step Functions	Serverless, event-driven, JSON state machine	AWS-only shops

6. MLflow — Experiment Tracking and Model Registry

6.1 Tracking

```
import mlflow

mlflow.set_tracking_uri("postgresql://mlflow:pass@rds-endpoint:5432/mlfl
mlflow.set_experiment("P053-DRAM-Yield")

with mlflow.start_run(run_name="day1-a100-v4"):
    # Parameters
    mlflow.log_params({
        "model": "HybridTransformerCNN",
        "epochs": 50,
        "lr": 1e-3,
        "amp_dtype": "bfloat16",
        "gpu": "A100-SXM4-40GB",
        "batch_size": 512,
    })

    # Metrics (per epoch)
    for epoch in range(50):
        mlflow.log_metrics({
            "train_loss": train_loss,
            "val_auc_pr": val_auc_pr,
            "val_auc_roc": val_auc_roc,
            "val_f1": val_f1,
        }, step=epoch)

    # Artifacts
    mlflow.log_artifact("best_model.pt")
    mlflow.log_artifact("training_curves.png")
    mlflow.log_artifact("confusion_matrix.png")
```

6.2 Model Registry

```
# Register model as a new version
model_uri = f"runs:/{run_id}/model"
mv = mlflow.register_model(model_uri, "P053-DRAM-Yield-Model")
# Creates version 1, 2, 3... automatically

# Set alias (like a pointer)
client = mlflow.tracking.MlflowClient()
client.set_registered_model_alias("P053-DRAM-Yield-Model", "champion", m

# Load production model
champion = mlflow.pyfunc.load_model("models:/P053-DRAM-Yield-Model@champ

# Rollback (point alias to previous version)
client.set_registered_model_alias("P053-DRAM-Yield-Model", "champion", "
# Instant rollback — no redeployment needed
```

6.3 P053 MLflow Architecture

P053 MLflow stack:

- **Tracking server:** MLflow container on EC2, connected to RDS PostgreSQL
- **Artifact store:** S3 (`s3://p053-mlflow-artifacts/mlflow/`)
- **Model Registry:** `@champion` and `@challenger` aliases for A/B testing
- **Retrain flow:** Drift detected → train on T4 GPU → register new version → canary test → promote @champion
- **Rollback flow:** Canary fails → revert @champion alias to previous version → log rollback event

7. Docker and Kubernetes for ML

7.1 Docker Compose

```
# P053 production stack (simplified from docker-compose-bigdata-aws.yml)
version: '3.8'
services:
  airflow-scheduler:
    image: p053-airflow-gpu:latest    # Custom image with PyTorch + CUDA
    runtime: nvidia                     # NVIDIA GPU access
    environment:
      NVIDIA_VISIBLE_DEVICES: all
      AWS_DEFAULT_REGION: us-west-2
    volumes:
      - ./dags:/opt/airflow/dags
      - ./src:/opt/airflow/src

  mlflow:
    image: ghcr.io/mlflow/mlflow:latest
    environment:
      MLFLOW_BACKEND_STORE_URI: postgresql://mlflow:pass@rds-endpoint/ml
      MLFLOW_DEFAULT_ARTIFACT_ROOT: s3://p053-mlflow-artifacts/mlflow/
    ports: ["5001:5000"]

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    environment:
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
      KAFKA_NUM_PARTITIONS: 4

  spark-master:
    image: bitnami/spark:3.5
    environment:
      SPARK_MODE: master

  api:
    image: p053-api:latest
    ports: ["8000:8000"]

  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
```

7.2 Kubernetes Concepts

Concept	What It Does
Pod	Smallest unit — one or more containers sharing network/storage
Deployment	Declares desired state: "run 3 replicas of this container"
Service	Stable network endpoint for a set of pods
HPA	Horizontal Pod Autoscaler — scales pods based on CPU/memory
ConfigMap	Store configuration (env vars, config files) separately from code
Secret	Store sensitive data (passwords, API keys) encrypted
Ingress	External HTTP routing to services
PVC	Persistent Volume Claim — attach storage to pods

7.3 NVIDIA Container Toolkit

```
# Install on EC2 (from ec2-user-data.sh)
distribution=$(. /etc/os-release; echo $ID$VERSION_ID)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gp
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Verify GPU access in Docker
docker run --gpus all nvidia/cuda:12.1-runtime nvidia-smi
# Should show Tesla T4, 16GB VRAM

# In docker-compose.yml:
# runtime: nvidia
# OR deploy.resources.reservations.devices with capabilities: [gpu]
```

8. Monitoring and Observability

8.1 Prometheus

Prometheus is a time-series metrics database. It **pulls** (scrapes) metrics from your services every 15 seconds.

```
# Python — expose metrics for Prometheus
from prometheus_client import Counter, Histogram, start_http_server

# Define metrics
PREDICTIONS = Counter('predictions_total', 'Total predictions', ['result
LATENCY = Histogram('prediction_latency_seconds', 'Prediction latency')

# In your API endpoint
@app.post("/predict")
async def predict(data):
    with LATENCY.time():
        result = model.predict(data)
    PREDICTIONS.labels(result='pass' if result > 0.5 else 'fail').inc()
    return {"prediction": result}

# Start metrics server on :8001
start_http_server(8001)
```

8.2 Grafana

Grafana visualizes Prometheus metrics in real-time dashboards.

> **P053 Grafana dashboards:**
>
> - **Inference dashboard:** Request rate, latency (p50/p95/p99), error rate
> - **Model performance:** AUC-PR over time, prediction distribution, confidence scores
> - **Drift monitoring:** PSI per feature, KL divergence, KS test p-values
> - **Infrastructure:** GPU utilization, CPU usage, memory, disk I/O

8.3 Model Drift Detection

Method	What It Measures	Threshold	P053 Usage
PSI	Population Stability Index — compares feature distributions	> 0.10 warning > 0.25 critical	✅ Primary drift metric
KL Divergence	Information loss when using new distribution vs reference	> 0.1	✅ Per-feature divergence
KS Test	Kolmogorov-Smirnov — max distance between CDFs	$p < 0.05$	✅ Statistical significance
ADWIN	Adaptive Windowing — detects concept drift in streams	Adaptive	❌ Not used (batch mode)

9. Data Pipeline Patterns

9.1 Batch vs Stream vs Lambda vs Kappa

Pattern	How It Works	P053?
Batch	Process data in scheduled intervals (hourly, daily)	✅ Daily simulation DAG
Stream	Process data as it arrives (Kafka → Spark Streaming)	✅ Kafka → Spark ingestion
Lambda	Batch + Stream in parallel (two codebases)	❌ Too complex
Kappa	Stream only (replay Kafka for batch-like processing)	Partially — could replay from Kafka

9.2 ETL vs ELT

Pattern	Flow	When to Use
ETL	Extract → Transform → Load (transform BEFORE storage)	Limited storage, need clean data. **P053 uses this.**
ELT	Extract → Load → Transform (transform AFTER storage)	Cheap storage (S3), flexible queries. Modern data warehouses.

9.3 Data Quality

Tool	Purpose	P053 Equivalent
Great Expectations	Data validation framework ("expect column X to not be null")	Custom validation in Spark ETL task
dbt	SQL-based data transformation + testing	Not used (Python pipeline)
Pandas Profiling	Auto-generate EDA reports	NB01 EDA notebook

10. Full Technology Comparison Matrix

Category	Technology	Used in P053?	Alternative
Streaming	Apache Kafka	✅	AWS Kinesis, RabbitMQ, Pulsar
	Kafka Connect	❌	Manual producers/consumers
	Kafka Streams	❌	Spark Structured Streaming
Processing	Apache Spark	✅	Dask, Ray, Polars
	Apache Flink	❌	Better for true streaming
	Pandas	✅ (EDA only)	Polars (faster)
Orchestration	Apache Airflow	✅	Prefect, Dagster, Argo
	AWS Step Functions	❌	Serverless alternative
ML Tracking	MLflow	✅	W&B, Neptune, ClearML
	TensorBoard	❌	PyTorch-native, simpler
Monitoring	Prometheus + Grafana	✅	Datadog, New Relic, CloudWatch
	ELK Stack	❌	Log analytics
Containers	Docker Compose	✅	Podman
	Kubernetes	✅ (manifests)	ECS, Docker Swarm
Storage	S3	✅	GCS, ADLS, MinIO
	PostgreSQL (RDS)	✅	MySQL, Aurora
	Redis	✅	Memcached, DynamoDB

11. P053 Complete Big Data Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          AWS g4dn.xlarge EC2                             │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  Docker Compose (docker-compose-bigdata-aws.yml)                 │    │
│  │                                                                  │    │
│  │  ┌──────────────┐    ┌──────────┐    ┌────────────────┐         │    │
│  │  │   Airflow    │──►│  Kafka   │──►│ Spark Master   │         │    │
│  │  │ (GPU sched.) │    │ (broker) │    │  + Executor    │         │    │
│  │  └──────┬───────┘    └──────────┘    └────────┬───────┘         │    │
│  │         │                                     │                 │    │
│  │         │  dag_retrain_pipeline.py            │                 │    │
│  │         │  (REAL GPU training on T4)          │                 │    │
│  │         ▼                                     ▼                 │    │
│  │  ┌──────────────┐    ┌──────────┐    ┌────────────────┐         │    │
│  │  │   MLflow     │    │ FastAPI  │    │  Prometheus    │         │    │
│  │  │  (tracking)  │    │ (serve)  │    │   + Grafana    │         │    │
│  │  └──────┬───────┘    └──────────┘    └────────────────┘         │    │
│  │         │                                                        │    │
│  └─────────┼────────────────────────────────────────────────────────┘    │
│            │                                                             │
│  ┌─────────▼──────────┐                  ┌────────────────────┐         │
│  │  T4 GPU (16GB)     │                  │ ec2_auto_stop.py   │         │
│  │  PyTorch 2.x       │                  │ CloudWatch $500    │         │
│  │  CUDA 12.1         │                  │ alarm              │         │
│  └────────────────────┘                  └────────────────────┘         │
└──────────────────────────────────────────────────────────────────────────┘
                   │                                  │
                   ▼                                  ▼
          ┌──────────────────┐              ┌──────────────────────┐
          │  S3 Bucket       │              │  RDS PostgreSQL      │
          │  - models/       │              │  - MLflow experiments │
          │  - data/         │              │  - MLflow runs       │
          │  - drift/        │              │  - MLflow registry   │
          │  - benchmarks/   │              │  db.t3.micro         │
          └──────────────────┘              └──────────────────────┘
```

12. Interview Questions with Answers

Q1: "What's the most complex data pipeline you've built?"

> **Answer:** "P053 — a 40-day production simulation processing 200M rows of DRAM sensor data. Kafka streams 5M-10M rows/day to Spark for ETL and feature engineering. Airflow orchestrates 3 DAGs: a master simulation controller, a daily pipeline (Kafka→Spark→predict→drift), and a retrain pipeline that triggers real GPU training on T4 when drift exceeds PSI>0.10. All on a single g4dn.xlarge instance with auto-stop protection. MLflow tracks everything with PostgreSQL backend and S3 artifacts. The pipeline automatically retrained the model 3 times across 40 days and auto-stopped the EC2 instance at completion."

Q2: "How do you handle model drift in production?"

> **Answer:** "Three-layer detection: PSI for distribution shift (>0.10 warning, >0.25 critical), KL divergence for information-theoretic drift, and KS test for statistical significance. When 3+ features cross critical thresholds AND model age > 7 days AND sufficient new data available (>50K rows), the retrain pipeline fires automatically. The new model goes through a canary test — we compare AUC-PR against the champion. If it improves, we promote via MLflow Model Registry alias swap. If not, we rollback the alias and log the failed attempt."

Q3: "Why Kafka instead of just batch processing?"

> **Answer:** "In a real semiconductor fab, sensor data arrives continuously from test stations — not in neat daily batches. Kafka gives us: (1) Decoupling — the data generator doesn't need to know about Spark or the model. (2) Replay — if drift detection has a bug, we can reprocess last 7 days from Kafka. (3) Real-time capability — we can switch from 5-minute micro-batches to true streaming if latency requirements tighten. (4) Multiple consumers — Spark reads for ETL, a monitoring service reads for alerting, both from the same topic."

Q4: "Why Spark instead of Pandas/Polars?"

> **Answer:** "I literally demonstrated pandas OOMing on 100M rows before introducing Spark. Pandas loads everything into RAM — on a g4dn.xlarge with 16GB, that's a 4-8GB limit. Our daily volume is 5-10M rows, and drift detection needs reference windows of 50M+ rows across multiple days. Spark processes this in partitions, spills to disk if needed, and scales horizontally if we add

> executors. Polars is faster than Pandas but still single-node — when your pipeline needs to grow to 100M+/day, Spark is the only answer."

Q5: "How do you decide between building vs buying infrastructure?"

> **Answer:** "Cost-benefit analysis at every layer. We use OPEN SOURCE (Kafka, Spark, Airflow, MLflow, Prometheus, Grafana) instead of managed services (AWS Kinesis, EMR, Step Functions, SageMaker) because: (1) No vendor lock-in — the same pipeline runs on AWS, GCP, or on-prem. (2) Cost — managed Kafka (MSK) costs $300/month for a minimal cluster vs $0 for open source in Docker. (3) Learning — I can debug Kafka internals, not debug a black box. The tradeoff is operational overhead, which we mitigate with Docker Compose and ec2-user-data.sh bootstrap."